

Computer Vision

Session 3

Agenda:

1	Different Type of CNN Models
2	Transfer learning

Different Type of CNN Models

Different Type of CNN Models

CNNs have evolved over time with various architectures designed to improve performance, accuracy, and efficiency for tasks like image classification and object detection. Let's go through some of the most well-known CNN architectures: **LeNet**, **AlexNet**, **VGG**, **Inception**, **ResNet**, and the concept of **Transfer Learning**.

Different Type of CNN Models

1. LeNet (LeNet-5)

- **LeNet-5** was developed by **Yann LeCun** in 1998 and is one of the earliest CNN architectures. It was primarily designed for **handwritten digit classification** on the **MNIST dataset**.



Different Type of CNN Models

1. LeNet (LeNet-5)

- The network is made up of **seven layers**, each with its own trainable parameters (including convolutional layers and fully connected layers).
- **Input Size:** It accepts input images of size **32 x 32 pixels**, which is larger than the standard 28x28 MNIST images, so padding is applied to the input.
- **Activation Function:** It uses **ReLU** (Rectified Linear Unit) as the activation function, which helps introduce non-linearity into the network.

Different Type of CNN Models

1. LeNet (LeNet-5)

LeNet's Design for Small Data

- LeNet was designed to recognize **handwritten digits** from 0 to 9. It is structured to work with small-scale images, and the dataset it was trained on was much smaller than modern datasets like ImageNet.

Layers

- The first convolutional layer uses 5x5 filters with a stride of 1.
- The second layer is a 2x2 average-pooling layer (also known as sub-sampling) with a stride of 2.
- The third layer is another convolutional layer with 16 filters.
- After additional pooling layers, the final layers are fully connected with 120 and 84 neurons, followed by an output layer of 10 neurons for classification.

Different Type of CNN Models

1. LeNet (LeNet-5)

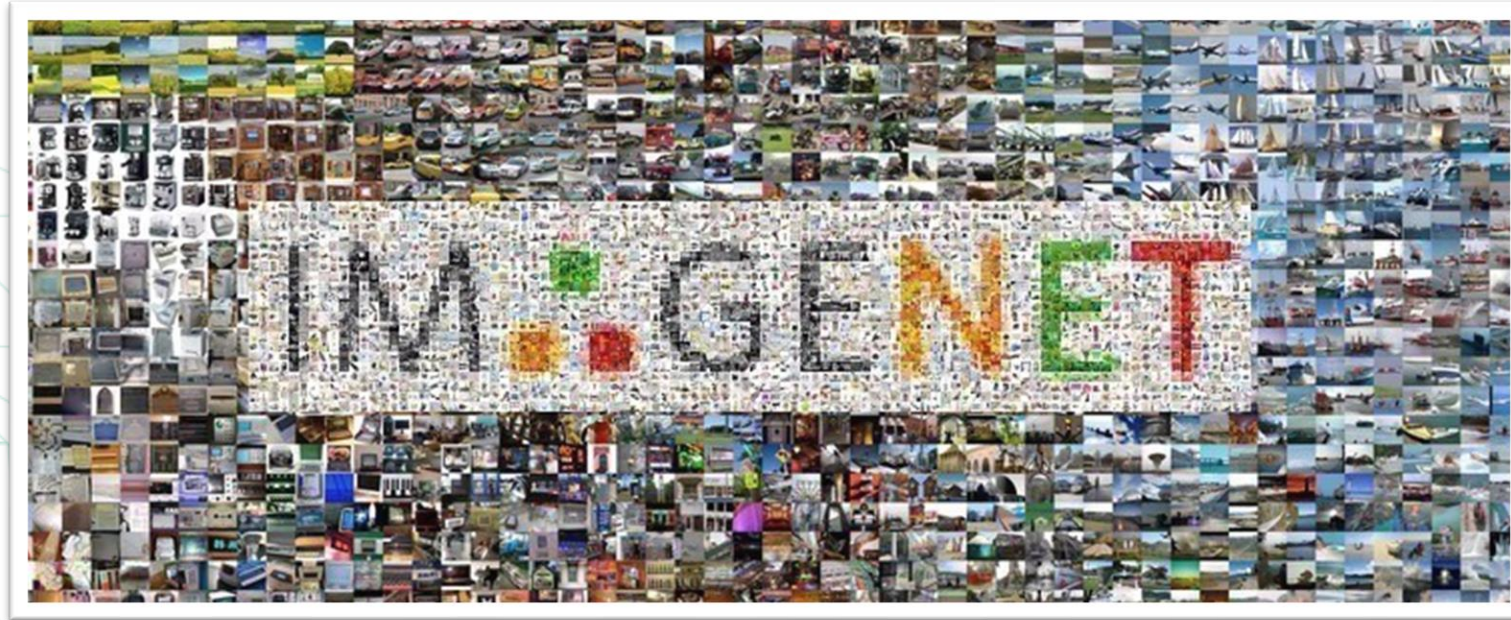
Performance and Limitations

- While LeNet was **effective** in classifying handwritten digits, it had **limitations** when it came to handling more complex tasks, especially when working with **larger images** or a **large number of object classes**.
- The **architecture** is simple and efficient for tasks involving small images and a limited number of classes, but it struggles with larger datasets due to its relatively shallow depth and small receptive field.

Different Type of CNN Models

2. AlexNet

- **AlexNet**, proposed in 2012, is similar to LeNet in concept but **deeper** and more **complex**, making it better suited for large-scale datasets like **ImageNet**.



Different Type of CNN Models

2. AlexNet Architecture

Improvements over LeNet

- **More Filters:** AlexNet uses a greater number of filters in its convolutional layers, enabling it to learn more complex and varied features.
- **Dropout Regularization:** Dropout is introduced in AlexNet to prevent **overfitting** by randomly dropping some connections during training.
- **ReLU Activation:** Used to introduce non-linearity and make the model more effective for complex tasks.

Different Type of CNN Models

2. AlexNet Architecture

Key Design Concepts

AlexNet introduced key ideas in CNN design, such as

- **Deeper Layers:** Five convolutional layers, followed by **max pooling** and three fully connected layers, enable the model to extract complex features from images.
- **Larger Filter Size:** The first convolutional layer in AlexNet uses a relatively large **11x11 filter**, which was a novel approach at the time.
- **Max Pooling:** Used after convolutional layers to reduce dimensionality and extract the most prominent features.

Different Type of CNN Models

2. AlexNet Architecture

Overcoming Overfitting

- AlexNet introduced **dropout layers** to solve the issue of overfitting, especially since the model had **60 million parameters**, making it prone to overfitting without regularization.
- Dropout randomly deactivates neurons during training, preventing the network from becoming overly reliant on certain patterns and improving generalization.

Different Type of CNN Models

2. AlexNet Architecture

Computational Challenges

- AlexNet's Limitations
 - Despite its success, AlexNet still had a relatively high number of parameters, requiring large memory and computation resources, especially during training. This was addressed with **GPU training**.
 - **Parameter Count:** The network had **55 million parameters**, most of which were in the **fully connected layers**, resulting in a heavy computational load.
 - Later models like **VGG** solved some of these issues by using smaller filters but increasing depth.

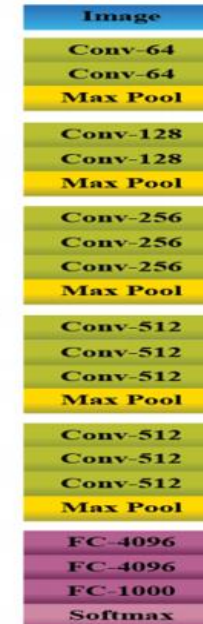
Different Type of CNN Models

3. VGG-16

- VGG-16 has **16 layers**: 13 convolutional layers followed by 3 fully connected layers, with max pooling layers in between.

VGG-16

16 layer
3 × 3 convolution



Different Type of CNN Models

3. VGG-16

Improvement over AlexNet

- VGG-16 significantly improved **accuracy** over AlexNet, achieving a top-5 accuracy of **92.7%** on the **ImageNet dataset**.
- VGG-16 became one of the most widely used models due to its simple and effective design.

Different Type of CNN Models

Key Architectural Features of VGG-16

3x3 Convolutional Filters

- One of the key changes VGG-16 introduced was the use of **multiple 3x3 filters** in place of larger filters like 5x5 or 7x7.
- Stacking two 3x3 filters in succession gives an effective receptive field size of 5x5, while using fewer parameters and less computation.
- This led to more efficient and deeper networks.

Different Type of CNN Models

Key Architectural Features of VGG-16

Parameter Efficiency

- By stacking **3x3 convolutions**, VGG-16 achieves the same receptive field size as a **5x5 filter** but with fewer parameters and lower computational cost.
- **Why this is important:** Larger filters like 5x5 introduce more parameters, increasing the computational burden. Stacking smaller filters reduces this burden while maintaining performance.

Different Type of CNN Models

VGG-19 Architecture

- **VGG-19** is an extension of VGG-16 with **19 layers**: 16 convolutional layers and 3 fully connected layers.

Difference from VGG-16

- VGG-19 simply extends the depth of the network, adding more convolutional layers (19 in total), but follows the same basic architecture as VGG-16.

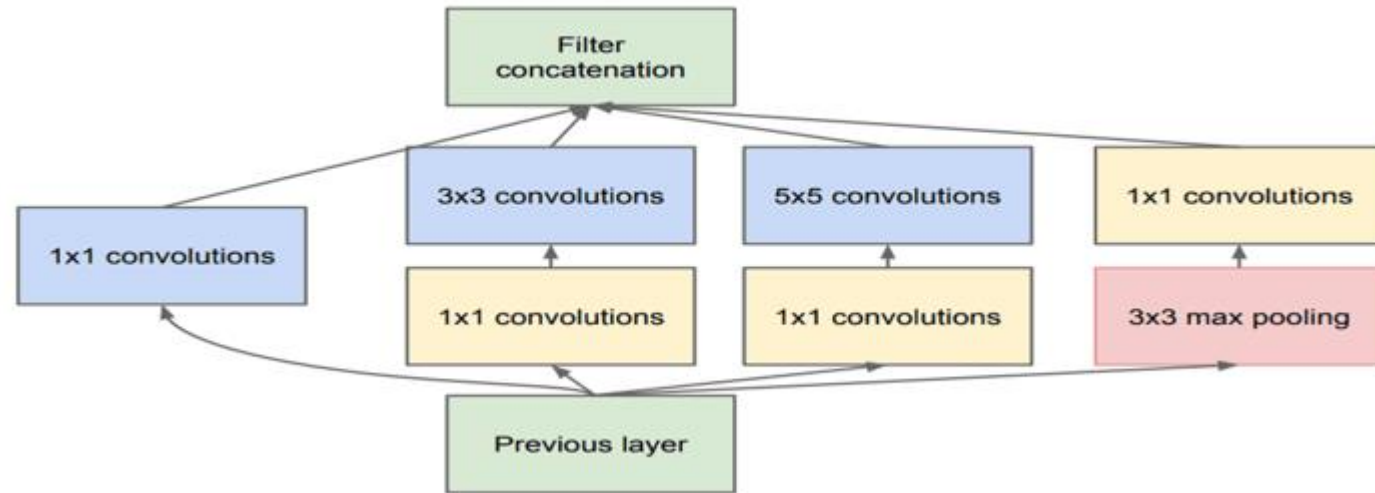
Use Case

- VGG-19 is often used in transfer learning tasks due to its high performance in feature extraction, especially for tasks like object detection, segmentation, and fine-tuned classification tasks.

Different Type of CNN Models

4. InceptionNet (GoogLeNet)

- InceptionNet consists of **22 layers** (or 27 if pooling layers are included) with **five pooling layers** interspersed throughout the network. The final layers use **Global Average Pooling** instead of fully connected layers.



Different Type of CNN Models

4. InceptionNet (GoogLeNet) Architecture

Key Applications

- It has been used for various tasks like **face detection and identification, adversarial training**, and image classification. Its ability to detect complex patterns made it a versatile model for many vision tasks.

Different Type of CNN Models

Inception Module

- The **Inception Module** is the core innovation of GoogLeNet. It allows the network to use multiple filter sizes (1x1, 3x3, 5x5) **simultaneously** for each input. This design allows the network to capture information at multiple scales:
 - **1x1 Convolution:** Used to reduce the number of input channels (depth) and computational cost.
 - **3x3 and 5x5 Convolutions:** Capture mid- and large-scale features.
 - **Max Pooling:** Used to reduce spatial dimensions while maintaining important features.
 - All these outputs are concatenated into one feature map, allowing the network to "decide" which size filter is the most useful.

Different Type of CNN Models

Inception Module

Advantage

- The inception module significantly reduces the computational cost by allowing different-sized filters to process information in parallel, improving feature extraction at multiple scales.

Different Type of CNN Models

Multiple Output Layers

- **Purpose:** To solve the vanishing gradient problem that can occur in deep networks, InceptionNet uses **multiple output layers**. This ensures that gradients flow more easily through the network, preventing issues where weights in earlier layers do not update properly due to the vanishing gradient problem.
- **Implementation:** These intermediate outputs are connected to auxiliary classifiers that help the network learn better representations. They also serve to smooth out the loss during training.
- **Vanishing Gradient Fix:** Having intermediate outputs helps maintain strong gradient signals during backpropagation, making it easier to train the network even with its deep architecture.

Different Type of CNN Models

Global Average Pooling (GAP)

- **What It Does:** Instead of using fully connected layers at the end of the network (like AlexNet or VGG), GoogLeNet uses **Global Average Pooling**:
 - This replaces the fully connected layers and averages the spatial dimensions (height and width) of each feature map, reducing each feature map to a single value.
 - The result is a much smaller output size (a vector of class scores) that is fed directly into the final **softmax layer** for classification.

Different Type of CNN Models

Global Average Pooling (GAP)

Why It's Efficient

- Fully connected layers add a significant number of parameters, especially with large feature maps. Global Average Pooling removes these layers, reducing the parameter count significantly.
- In comparison to VGG, which used 4.7 million parameters in the fully connected layers, GoogLeNet only requires around **200,000 parameters** in the final layers due to GAP.
- **Benefit:** This drastic reduction in parameters decreases computational cost and reduces the risk of overfitting, while still maintaining high classification accuracy.

Different Type of CNN Models

Key Advantages of InceptionNet (GoogLeNet)

- **Multi-Scale Feature Extraction:** The **Inception Module** uses filters of different sizes to capture features at multiple scales within the image.
- **Reduced Parameters:** By using 1x1 convolutions and replacing fully connected layers with **Global Average Pooling**, GoogLeNet drastically reduces the number of parameters compared to earlier architectures like VGG.
- **Multiple Output Layers:** Helps with training deep networks by mitigating the vanishing gradient problem.
- **Efficiency:** Despite its depth (22-27 layers), GoogLeNet is computationally efficient and requires fewer parameters, making it suitable for large-scale image classification tasks.

Different Type of CNN Models

Solving the Vanishing Gradient Problem with Skip Connections (ResNet)

- In **ResNet**, the **vanishing gradient problem** is addressed by introducing what is known as **skip connections** (also called **residual connections**). This is different from **GoogLeNet**, which used multiple outputs to mitigate this issue.

Different Type of CNN Models

Solving the Vanishing Gradient Problem with Skip Connections (ResNet)

How it works ?

- Instead of allowing the output of each layer to flow directly into the next layer, **skip connections** "skip" over one or more layers. The input is passed to the next layer **and** directly to a layer further downstream.
- The idea is that if deeper layers struggle to learn new features, the skip connection allows the network to simply pass the original information directly to later layers.
- This technique ensures that during **backpropagation**, the gradients can bypass some layers and flow back through the network more easily, preventing them from becoming too small (vanishing gradient).

Different Type of CNN Models

Residual Learning with Skip Connections

- In **ResNet**, the layers are designed to learn a **residual function** rather than the full transformation function. The goal of residual learning is to make it easier for the network to learn incremental changes, especially as the network gets deeper.

What it does:

- Each residual block contains two or more layers and a **skip connection** that bypasses these layers. The network doesn't need to learn everything from scratch in every block—it can focus on the residual (the difference between the input and the desired output of that block).
- **Residual connections** help the network learn effectively even when the model is very deep (e.g., **ResNet-50** or **ResNet-101**).

Different Type of CNN Models

How ResNet Differs from InceptionNet

- **InceptionNet** stacks layers of convolutions with various filter sizes (1x1, 3x3, 5x5) to capture information at different scales. The layers "pass information to each other" without skipping any steps.
- In contrast, **ResNet** introduces **residual connections**, where the output of a block is added to the input before it is passed to the next block.
 - This means that each layer can directly access information from previous layers through the **skip connections**, enabling the network to learn residuals rather than the entire transformation.

Transfer learning

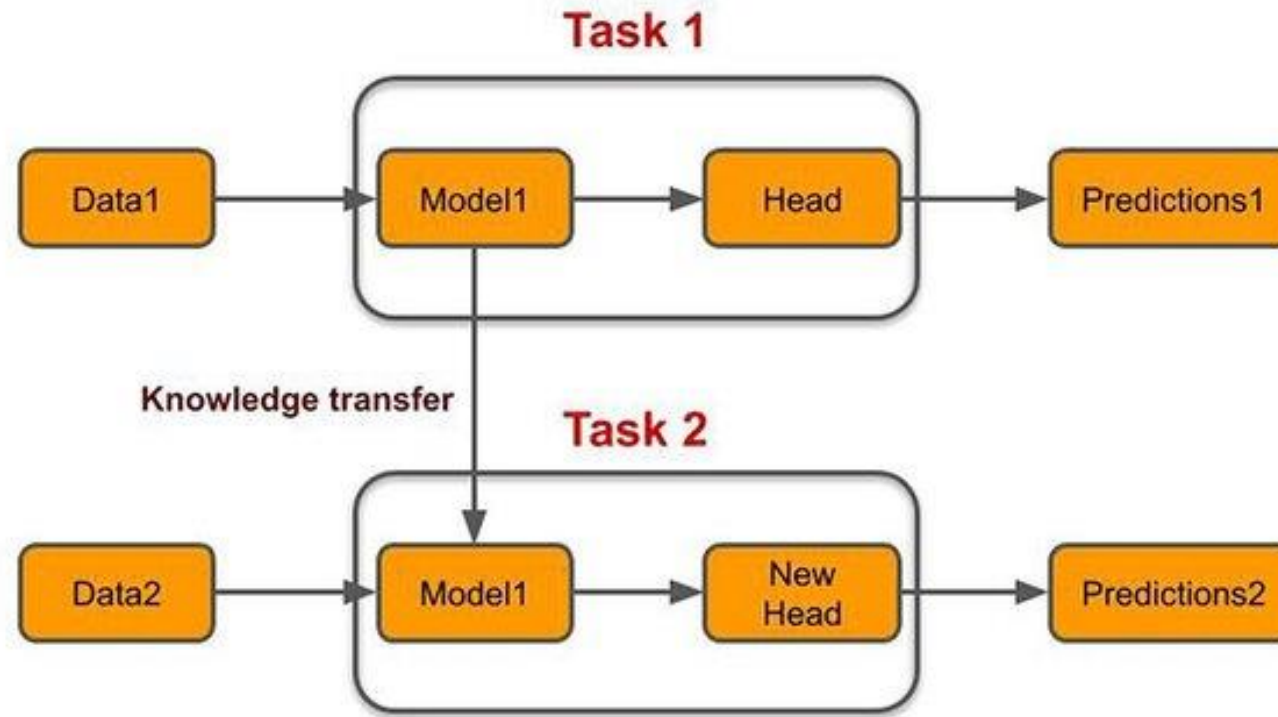
Transfer learning

What is Transfer Learning?

- **Transfer learning** is a machine learning technique where a model trained on one task is reused as the starting point for a model on a second task.
- The main idea is to **leverage pre-trained models** (such as models trained on large datasets like ImageNet) and adapt them to new tasks that might have smaller datasets or different classification goals.
- **Example:** If a model is pre-trained to classify images into 1000 categories, you can use the same model and adapt it to classify different objects or fewer categories by reusing its pre-trained weights.

Transfer learning

What is Transfer Learning?



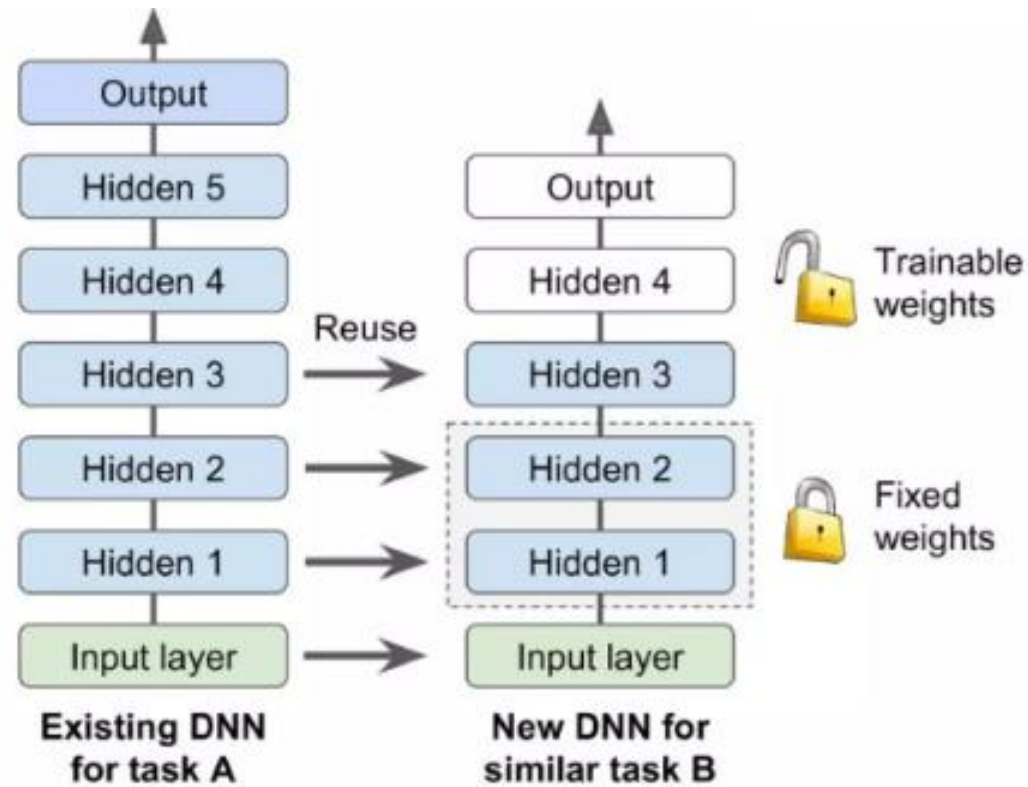
Transfer learning

Steps in Transfer Learning

- **Pre-trained Model:** Start with a model that has been trained on a large dataset (like ImageNet) and has learned meaningful features (edges, textures, shapes) from that dataset.
- **Freezing Layers:** In transfer learning, the lower layers (which have learned basic image features) are often **frozen**, meaning their weights are not updated. These layers are retained as they are already effective at extracting low-level features.
- **Tweaking Higher Layers:** The **upper layers** (closer to the output) are usually **tweaked** or replaced with new layers, depending on the task. These layers are trained with the new data to adapt to the specific categories or tasks required by the new problem.

Transfer learning

Steps in Transfer Learning



Transfer learning

Benefits of Transfer Learning

- **Time and Resource Efficiency:** Training a model from scratch requires a large dataset and significant computation resources. Transfer learning allows you to use a pre-trained model and fine-tune it on your specific dataset, saving time.

Transfer learning

Benefits of Transfer Learning

- **Better Performance on Small Datasets:** If you have a small dataset, training a deep network from scratch might lead to overfitting. Using a pre-trained model allows you to leverage the learned features from a larger dataset, improving generalization.

Transfer learning

Example of Transfer Learning

- A deep neural network (**DNN**) trained to classify images into 100 categories can be **adapted** to classify similar images into different categories (e.g., classifying vehicle types).
- **Freezing Lower Layers:** The lower layers (which have learned generic features like edges and textures) are kept **fixed** (frozen).
- **Replacing Upper Layers:** The upper layers are replaced or retrained to adapt the network to the new classification task.